

A convolução discreta na FPGA do Morphe

Roteiro prático — você escreve o módulo; o resto já está pronto

Antonio Nicassio Santos Lima Estágio LabPS/UEFS — supervisão: Prof. Armando S. Sanca

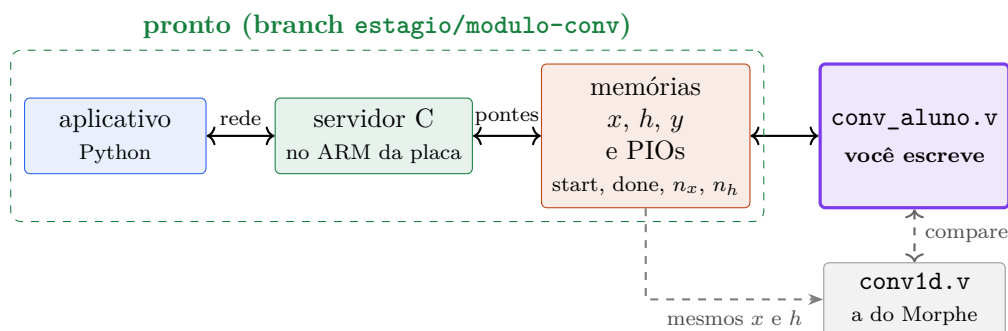
outubro de 2026

Neste roteiro você escreve, em Verilog, um bloco de hardware que calcula a **convolução discreta**

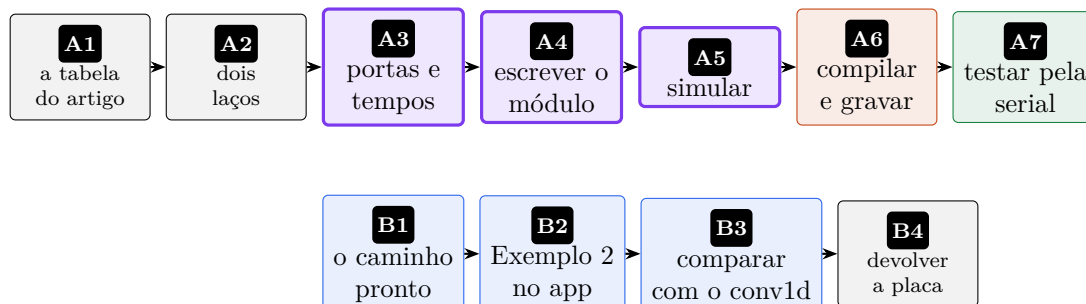
$$y[n] = x[n] * h[n] = \sum_k x[k] h[n - k]$$

e o compara com a convolução que o Morphe já tem na FPGA (o `conv1d.v`). A teoria vem do artigo *Um exemplo didático para o ensino da convolução discreta* (A. Ojeda González e I. C. P. Lamin, *Revista Univap*, v. 25, n. 49, p. 13–24, dez. 2019): o método da **tabela** que ele propõe é, quase sem mudança, o circuito que você vai construir.

Tudo em volta do seu módulo já está pronto. Você só escreve o corpo do `conv_aluno.v`:



- **Parte A** — construir o módulo: da tabela do artigo aos dois laços, das portas à máquina de estados; simular, compilar e testar **sozinho**, pela serial da placa, com o Exemplo 2 do artigo.
- **Parte B** — usar no Morphe: a janela **Convolução do aluno** manda os mesmos x e h para o seu módulo e para o `conv1d`, e mostra a diferença amostra a amostra.



Antes de começar

Faça

Na estação ou num computador do laboratório, num terminal (digite **bash** primeiro: em **ksh** os comandos do Quartus não são encontrados). Um clone **seu**, na sua pasta, um comando por vez:

```
cd ~
```

```
git clone -b estagio/modulo-conv https://github.com/nicassiosantos/morphe.git minha-conv
```

```
cd minha-conv
```

```
git switch -c minha-conv
```

O que é e por quê

O `/opt/morphe` é a cópia que a turma usa: mexer nela muda o Morphe de todos. O clone na sua pasta é só seu, e a branch nova guarda o seu trabalho separado. A branch `estagio/modulo-conv` já tem tudo o que este roteiro chama de “pronto”.

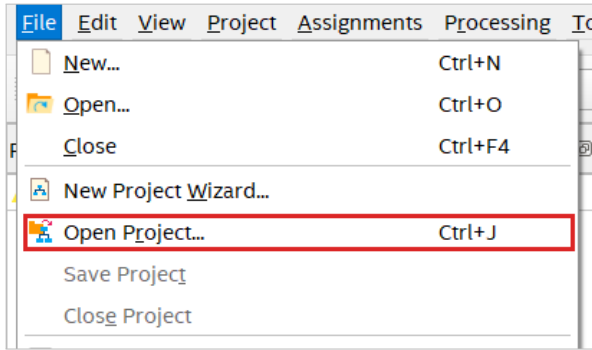
Cuidado

Copiar código do PDF pode juntar as linhas. Os mesmos trechos, em texto puro e na mesma ordem, estão em `docs/roteiro-conv/trechos.txt`.

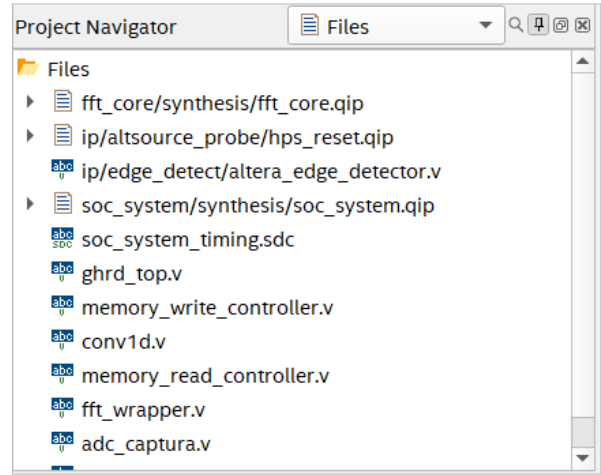
```
minha-conv/
├── Quartus/ ..... FPGA — abrir no Quartus 20.1
│   ├── conv_aluno.v ..... VOCÊ ESCRIVE — só as portas; o corpo é seu (A4)
│   ├── testbench/tb_conv_aluno.v ..... PRONTO — a simulação da A5
│   ├── conv1d.v ..... PRONTO — a convolução do Morphe, para comparar
│   ├── soc_system.qsys ..... PRONTO — memórias e PIOs do conv_aluno
│   ├── ghrd_top.v ..... PRONTO — o conv_aluno já ligado a eles
│   └── soc_system/, output_files/ ..... GERADO (não editar) na A6
├── C/ ..... servidor no ARM da placa
│   ├── morphe_server.c ..... PRONTO — opcode CONV_ALUNO
│   └── hps_0.h ..... GERADO (não editar) na A6 — os endereços
├── Python/ ..... aplicativo
│   └── conv_aluno_window.py ..... PRONTO — a janela da Parte B
└── docs/roteiro-conv/ ..... este roteiro e o trechos.txt
```

Faça

Abra o projeto no **Quartus 20.1** (menu de aplicativos, **Quartus Prime Lite 20.1**): **File** → **Open Project...** → `~/minha-conv/Quartus/soc_system.qpf`. No *Project Navigator*, troque **Hierarchy** por **Files** e dê duplo clique em `conv_aluno.v`.



File → Open Project...



Files: os arquivos do projeto

Cuidado

Sempre o 20.1. Se aparecer *IP upgrade recommended* ou *Used ... 23.1 (instead of 20.1)*, você abriu outra versão: feche **sem salvar**.

Parte A — construir o módulo

A1 A convolução pela tabela do artigo

O artigo resolve o seu **Exemplo 2** montando uma tabela: na linha de cima, $x[k]$; em cada linha de baixo, $h[n - k]$ — o h *invertido e deslizado* n casas para a direita. Multiplica coluna a coluna e soma a linha: o resultado é $y[n]$.

Na FPGA não há índice negativo: a memória começa no endereço 0. Por isso guardamos os sinais a partir do 0 e a origem do tempo fica por conta do aplicativo (B2). No Exemplo 2, $x = \{1, 2, 3, 1\}$ e $h = \{1, 2, 1, -1\}$:

$x[k]$	$k = 0$	$k = 1$	$k = 2$	$k = 3$		$k_{\min}$	$k_{\max}$
	1	2	3	1			
$n = 0: h[0 - k]$	1	0	0	0	$y[0] = 1 \cdot 1 = \mathbf{1}$	0	0
$n = 1: h[1 - k]$	2	1	0	0	$y[1] = 2 + 2 = \mathbf{4}$	0	1
$n = 2: h[2 - k]$	1	2	1	0	$y[2] = 1 + 4 + 3 = \mathbf{8}$	0	2
$n = 3: h[3 - k]$	-1	1	2	1	$y[3] = -1 + 2 + 6 + 1 = \mathbf{8}$	0	3
$n = 4: h[4 - k]$	0	-1	1	2	$y[4] = -2 + 3 + 2 = \mathbf{3}$	1	3
$n = 5: h[5 - k]$	0	0	-1	1	$y[5] = -3 + 1 = \mathbf{-2}$	2	3
$n = 6: h[6 - k]$	0	0	0	-1	$y[6] = \mathbf{-1}$	3	3

O resultado, $\{1, 4, 8, 8, 3, -2, -1\}$, é o da Equação (17) do artigo. Lá, h começa em $n = -1$, então y também começa em $n = -1$; aqui, tudo começa em 0. Os valores são os mesmos, só a origem do eixo muda.

O que é e por quê

Duas coisas da tabela viram hardware:

- **as casas em cinza não custam nada:** lá $x[k]$ ou $h[n-k]$ não existe, e o produto é zero. O artigo também as descarta (é o “intervalo que exclui os produtos iguais a zero”). Só as casas coloridas, de $k_{\min}$ a $k_{\max}$, precisam de conta;
- **uma linha de cada vez:** cada linha só precisa de um acumulador. Somou a linha, gravou $y[n]$, zero e passa para a próxima.

São $n_x + n_h - 1$ linhas: $4 + 4 - 1 = 7$ no exemplo.

A2 Da tabela a dois laços

As casas coloridas da linha n são as colunas onde $x[k]$ e $h[n-k]$ existem ao mesmo tempo:

$$\underbrace{0 \leq k \leq n_x - 1}_{x[k] \text{ existe}} \quad \text{e} \quad \underbrace{0 \leq n - k \leq n_h - 1}_{h[n-k] \text{ existe}} \iff n - n_h + 1 \leq k \leq n$$

$$k_{\min} = \max(0, n - n_h + 1) \quad k_{\max} = \min(n, n_x - 1)$$

Confira nas duas últimas colunas da tabela. Com isso, a tabela inteira é:

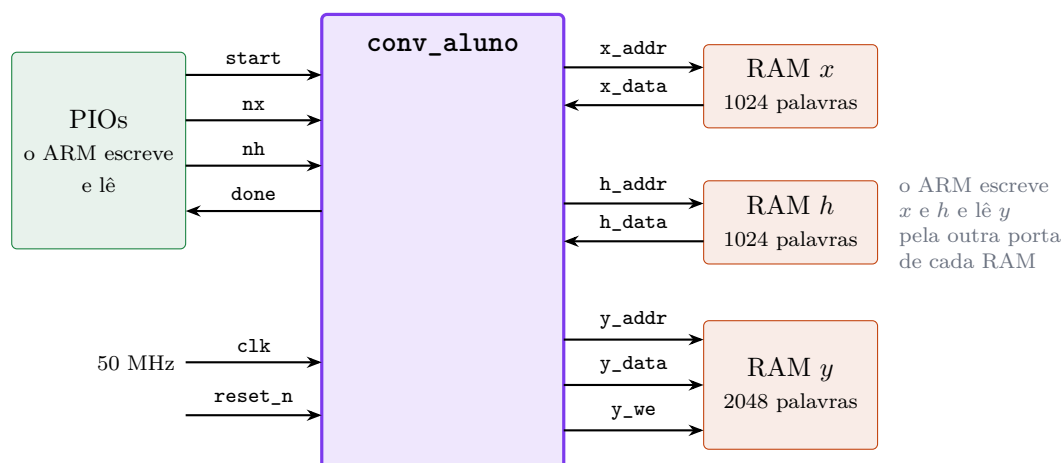
```
para n = 0 até nx + nh - 2:      # uma LINHA da tabela
  acc = 0
  para k = kmin(n) até kmax(n):  # as casas coloridas
    acc = acc + x[k] * h[n - k]
  y[n] = acc                     # a soma da linha
```

Pense

Faça a tabela com $x = \{2, 1\}$ e $h = \{1, 1, 1\}$ ($n_x = 2$, $n_h = 3$). Quantas linhas? Quais $k_{\min}$ e $k_{\max}$ em cada uma? Em alguma linha $k_{\min} > k_{\max}$? (Resposta: 4 linhas, $y = \{2, 3, 3, 1\}$; nunca, porque toda linha tem pelo menos uma casa colorida.)

A3 O que entra e o que sai do módulo

O `conv_aluno.v` já tem as portas. Não mude nomes nem larguras: é por eles que o topo (`ghrd_top.v`) o liga às memórias e aos PIOs.



```
module conv_aluno #(
  parameter DATA_WIDTH = 32,
  parameter X_ADDR_BITS = 10,    // x: ate 1024 amostras
```

```

parameter H_ADDR_BITS = 10,      // h: ate 1024 amostras
parameter Y_ADDR_BITS = 11      // y: ate 2047 amostras (nx + nh - 1)
)(
    input wire          clk,      // 50 MHz
    input wire          reset_n,  // ativo em 0

    // --- controle (PIOs) ---
    input wire          start,    // HPS -> FPGA
    output reg          done,     // FPGA -> HPS
    input wire [X_ADDR_BITS:0] nx, // amostras de x, 1 a 1024
    input wire [H_ADDR_BITS:0] nh, // amostras de h, 1 a 1024

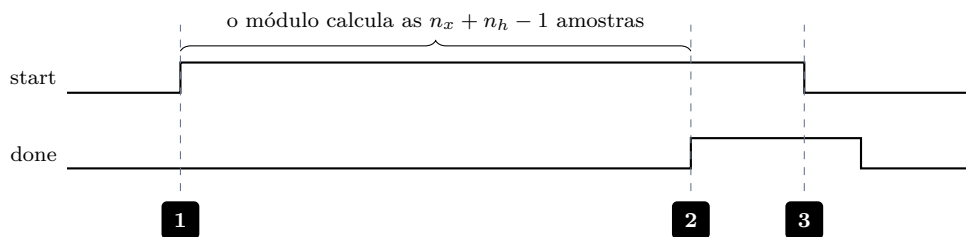
    // --- memoria de x[k] (so leitura) ---
    output reg [X_ADDR_BITS-1:0] x_addr,
    input wire signed [DATA_WIDTH-1:0] x_data,

    // --- memoria de h[k] (so leitura) ---
    output reg [H_ADDR_BITS-1:0] h_addr,
    input wire signed [DATA_WIDTH-1:0] h_data,

    // --- memoria de y[n] (so escrita) ---
    output reg [Y_ADDR_BITS-1:0] y_addr,
    output reg signed [DATA_WIDTH-1:0] y_data,
    output reg                  y_we
);

```

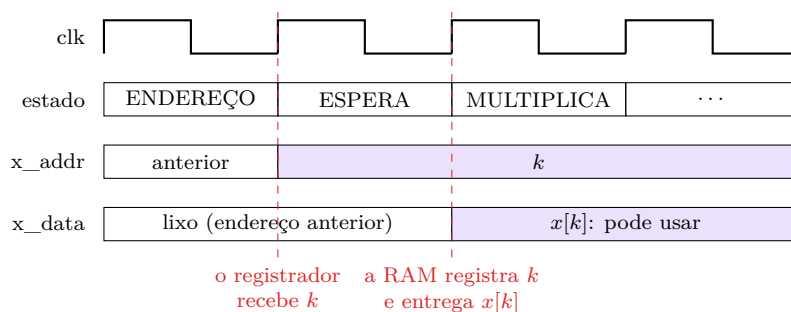
O start e o done



- 1** o ARM escreve x , h , n_x e n_h e sobe o **start**: é a **borda de subida** que dispara (guarde o **start** do ciclo anterior num registrador e compare);
- 2** terminou de gravar y : **done** = 1, e fica em 1;
- 3** o ARM leu y e baixou o **start**: **done** volta a 0 e o módulo espera o próximo **start**.

Ler a memória leva um ciclo

A memória é **síncrona**: ela registra o endereço na borda do relógio e só entrega o dado depois dela. Se o **x_addr** é um registrador que você muda num estado, o dado certo só pode ser usado **dois estados depois** — por isso a **ESPERA** no meio:

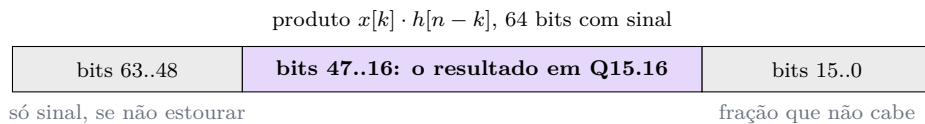


A escrita de y é mais simples: no ciclo em que $y_we = 1$, a memória grava y_data em y_addr . Ponha os três no mesmo estado e baixe o y_we no seguinte.

Multiplicar em Q15.16

O que é e por quê

A FPGA não tem ponto flutuante. O Morphe guarda cada número real como um inteiro de 32 bits, o valor vezes 2^{16} (**Q15.16**): $1,0 = 0x00010000$, $-1,0 = 0xFFFF0000$. Na soma, a vírgula não mexe. No **produto**, os dois fatores trazem 2^{16} cada, e o resultado sai com 2^{32} : 32 bits de fração. Para voltar a Q15.16, jogue fora os 16 bits de baixo e fique com os 32 seguintes.



Cuidado

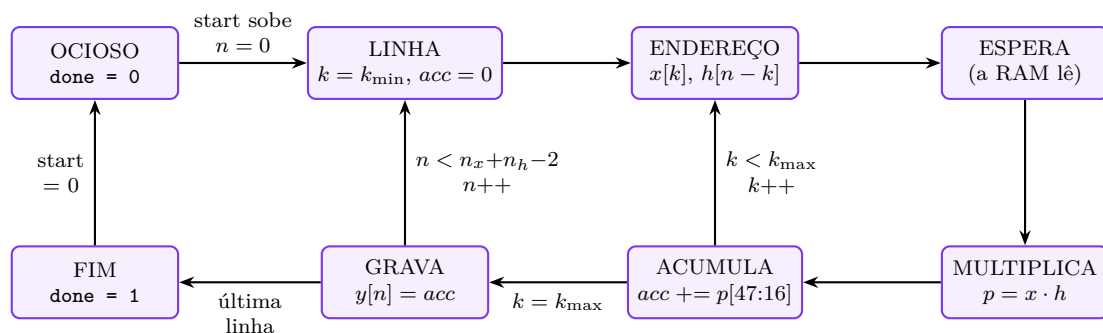
Para bater bit a bit com o conv1d (a Parte B compara *igualdade*, não “parecido”), faça a conta como ele faz:

- cada produto em 64 bits, com sinal: x_data e h_data já são **signed**, e o registrador que recebe o produto também precisa ser (**reg signed [63:0]**);
- de **cada** produto, os bits [47:16]; some esses valores, não os produtos inteiros;
- acumulador de **32 bits**, sem saturação.

Somar os 64 bits e cortar só no fim é até mais preciso, mas dá diferenças de 1 ou 2 unidades na última casa em relação ao conv1d.

A4 Escrever o módulo

Os dois laços da A2 viram uma **máquina de estados**: um estado faz uma coisa por ciclo de relógio, e o **case** decide o próximo. Um caminho possível (há outros):



Na tabela	Na máquina
começar uma linha	LINHA: k e $k_{\max}$ da linha n , acumulador zerado
uma casa colorida	ENDEREÇO → ESPERA → MULTIPLICA → ACUMULA: 4 ciclos
somar a linha	é o próprio acumulador, casa a casa
escrever $y[n]$	GRAVA: $y_addr = n$, $y_data = acc$, $y_we = 1$

Faça

No `conv_aluno.v`, entre o `)`; das portas e o `endmodule`, escreva:

1. os estados (`localparam`) e o registrador de estado;
2. os registradores n , k , $k_{\max}$, o acumulador e o produto. Cuidado com a largura e o sinal: n vai até 2046 e $n - n_h + 1$ pode ser **negativo** — use índices **signed** de 13 bits, por exemplo;
3. a detecção da borda do `start`;
4. um `always @(posedge clk or negedge reset_n)` com o reset (tudo em zero, estado OCIOSO) e o `case` dos estados.

Salve (Ctrl+S). Antes de compilar, simule (A5).

Cuidado

- $h[n - k]$: o endereço é $n - k$, calculado em largura de índice e só depois cortado para os 10 bits de `h_addr`.
- `y_we` só pode ficar em 1 no ciclo de gravar. Um jeito simples é pôr `y_we <= 0` no começo do `always` e `y_we <= 1` só no estado GRAVA.
- O módulo tem de **voltar** ao OCIOSO: o servidor roda uma convolução atrás da outra.

A5 Simular antes de compilar

O que é e por quê

Compilar o projeto leva de 10 a 15 minutos; simular leva um segundo. O `testbench/tb_conv_aluno.v` imita as três memórias (com o mesmo ciclo de leitura da A3), dispara o seu módulo como o servidor faz e compara cada $y[n]$ com a conta feita do jeito do `conv1d`. São sete casos: o Exemplo 2, uma amostra de cada lado, h maior que x , valores fracionários quaisquer, os dois extremos de 1024 amostras e uma segunda rodada seguida.

Faça

De dentro de `~/minha-conv/Quartus`:

```
iverilog -g2005 -o /tmp/tb_conv.vvp testbench/tb_conv_aluno.v conv_aluno.v
```

```
vvp /tmp/tb_conv.vvp
```

Confira

Cada caso numa linha começando com `ok`, e a última linha é TUDO OK:

```
ok    Exemplo 2 do artigo (nx=4 nh=4, 81 ciclos)
ok    1 x 1, fracao e sinal (nx=1 nh=1, 9 ciclos)
ok    h maior que x (nx=3 nh=6, 91 ciclos)
ok    aleatorio 50 x 17 (nx=50 nh=17, 3535 ciclos)
ok    x com 1024 amostras (nx=1024 nh=3, 14343 ciclos)
ok    h com 1024 amostras (nx=3 nh=1024, 14343 ciclos)
ok    segunda rodada seguida (nx=4 nh=4, 81 ciclos)
TUDO OK
```

O número de ciclos do seu pode ser outro: depende de quantos estados você usou. O que tem de bater são os valores.

O testbench diz	Provável causa
done nao subiu	o módulo não sai de um estado, ou não detecta a borda do start
done nao voltou a 0	falta voltar ao OCIOSO quando o start cai
valores deslocados ($y[n]$ tem a conta de outra casa)	usou o dado um estado cedo demais: falta a ESPERA
erra só nas primeiras ou nas últimas linhas	$k_{\min}$ ou $k_{\max}$ errado; índice sem sinal com $n - n_h + 1 < 0$
diferença de 1 na última casa	cortou os bits no fim, e não em cada produto
escrito fora da saida	gravou mais de $n_x + n_h - 1$ amostras, ou y_we ficou em 1
Exemplo 2: y nao e ...	a conta está errada até no caso do artigo: refaça a A1 à mão

Cuidado

Se o computador não tiver o iverilog, peça ao responsável pelo laboratório. Sem simular, cada erro custa uma compilação inteira.

A6 Compilar e gravar a placa

Já está pronto

As memórias `conv_aluno_x`, `_h`, `_y` e os PIOs `conv_aluno_start`, `_done`, `_nx`, `_nh` já estão no `soc_system.qsys`, com endereço travado; o `ghrd_top.v` já os liga ao seu módulo. Faltava só **gerar** o sistema uma vez e compilar.

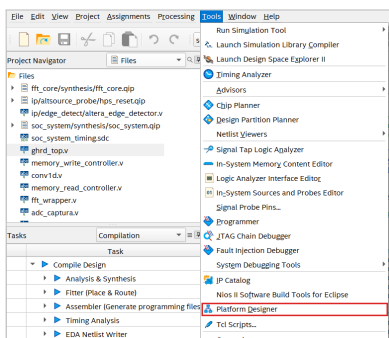
Faça

- Tools** → **Platform Designer** → `soc_system.qsys` → **Generate** → **Generate HDL...** → **Generate**. Espere *Generate Completed*, **Finish** e feche o Platform Designer **sem** mudar nada.
- No Quartus: **Processing** → **Start Compilation** (Ctrl+L).
- No terminal, de `~/minha-conv/Quartus`:

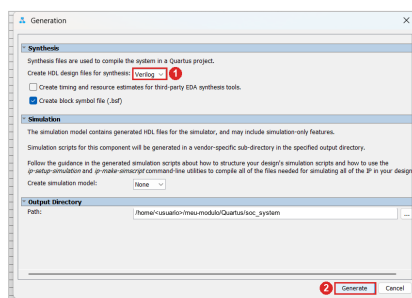
```
python3 gen_hps_header.py
```

- Na **estação**, de `~/minha-conv`, fora do horário de aula. No lugar de **CABO**, o nome do cabo da sua placa (o `jtagconfig` lista; se houver mais de um, desligue o USB-Blaster da sua placa, rode de novo e veja qual sumiu):

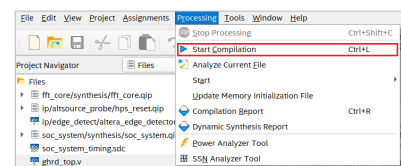
```
./morphe-up.sh --cable 'CABO' --deploy
```



Tools → Platform Designer



Generate HDL



Start Compilation

O que é e por quê

Gerar escreve o Verilog do sistema e o **.sopcinfo**. **Compilar** junta isso ao seu módulo e produz o **.sof**, a configuração da FPGA. O **gen_hps_header.py** tira os endereços do **.sopcinfo** para o **C/hps_0.h**, e o **--deploy** recompila o servidor da placa com ele: servidor e FPGA sempre andam juntos.

Confira

- *Full Compilation was successful*, e no **Compilation Report** → **Timing Analyzer**, *Slack* positivo no `clock_50_1`;
- `grep CONV_ALUNO ../C/hps_0.h` mostra `0x43000`, `0x44000`, `0x46000` e `0x180` a `0x1b0`;
- o `morphe-up.sh` termina com os testes 4/4.

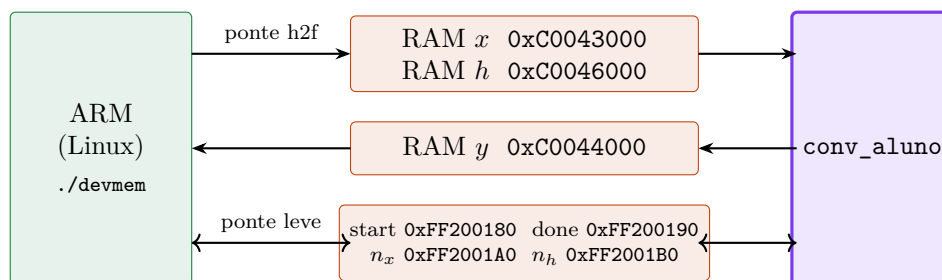
Cuidado

Erro de compilação que aponta para o `conv_aluno.v` é seu: o *Messages*, embaixo, diz a linha. O Quartus é mais rigoroso que o `iverilog` com larguras; leia os *Warnings* do seu arquivo também.

A7 Testar pela serial, com o Exemplo 2

O que é e por quê

O teste mais direto: você escreve x e h nas memórias, dá o **start** e lê y , digitando endereços pelo console da placa. Nenhum programa do Morphe no meio: se der errado, o erro está no seu módulo.



Faça

Entre na placa pela serial (a USB marcada UART; descubra o nome com `ls /dev/ttyUSB*` antes e depois de ligar o cabo) e abra o console, trocando `/dev/ttyUSB0` pelo nome da sua:

```
sudo screen /dev/ttyUSB0 115200
```

Usuário `root`. Dentro da placa, um comando por vez:

```
cd ~/morphe
```

1. Repouso, e os tamanhos ($n_x = n_h = 4$):

```
./devmem 0xFF200180 32 0
```

```
./devmem 0xFF2001A0 32 4
```

```
./devmem 0xFF2001B0 32 4
```

2. $x = \{1, 2, 3, 1\}$, uma palavra a cada 4 bytes:

```
./devmem 0xC0043000 32 0x00010000
```

```
./devmem 0xC0043004 32 0x00020000
```

```
./devmem 0xC0043008 32 0x00030000
```

```
./devmem 0xC004300C 32 0x00010000
```

3. $h = \{1, 2, 1, -1\}$:

```
./devmem 0xC0046000 32 0x00010000
```

```
./devmem 0xC0046004 32 0x00020000
```

```
./devmem 0xC0046008 32 0x00010000
```

```
./devmem 0xC004600C 32 0xFFFF0000
```

4. Disparar e conferir o done (tem de responder 0x00000001):

```
./devmem 0xFF200180 32 1
```

```
./devmem 0xFF200190
```

5. Ler $y[0]$ a $y[6]$:

```
for a in 00 04 08 0C 10 14 18; do ./devmem 0xC00440$a; done
```

6. Voltar ao repouso (e o done volta a 0x00000000):

```
./devmem 0xFF200180 32 0
```

	endereço	valor	o devmem responde
$y[0]$	0xC0044000	1	0x00010000
$y[1]$	0xC0044004	4	0x00040000
$y[2]$	0xC0044008	8	0x00080000
$y[3]$	0xC004400C	8	0x00080000
$y[4]$	0xC0044010	3	0x00030000
$y[5]$	0xC0044014	-2	0xFFFE0000
$y[6]$	0xC0044018	-1	0xFFFF0000

Confira

Os sete valores batem com a tabela da A1 e com a Equação (17) do artigo.

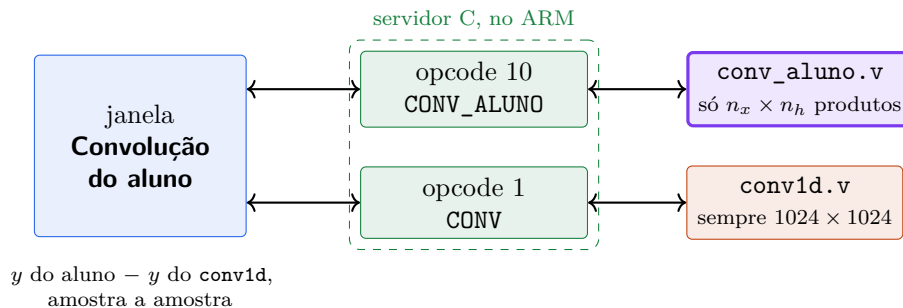
Cuidado

- Só use o **devmem** nesses endereços **depois** de gravar a placa com o seu **.sof** (A6). Num endereço sem componente, a placa **trava** e só volta desligando e ligando.
- **done** em 0x00000000 depois do passo 4: o módulo não terminou (ou nem começou). Volte à A5.
- Sem login na serial? Entre pela rede (**ssh root@IP da placa**) e use os mesmos comandos.

Parte B — usar no Morphe e comparar com o conv1d

B1 O caminho que já está pronto

Na A7 você fez à mão o que o servidor faz em cada pedido. Na Parte B, o aplicativo manda os **mesmos** x e h por dois caminhos e compara as respostas:



Peça (pronta)	O que faz
Python/conv_aluno_window.py	a janela: gera x e h , passa para Q15.16, faz os dois pedidos numa <i>thread</i> e desenha y e a diferença
Python/morphe_protocol.py	OP_CONV_ALUNO = 10: o pedido tem o mesmo formato da convolução (cabeçalho, x , depois h)
C/morphe_server.c: handle_conv_aluno	os passos da A7: escreve x , h , n_x , n_h , zera y , sobe o start , espera o done (até 5s), lê y , baixa o start e responde
C/morphe_server.c: conferência do <i>sysid</i>	recusa o pedido se o .sof da placa não for o que tem o conv_aluno: a placa não trava
Quartus/ghrd_top.v	liga as portas do seu módulo às memórias (chipselct e clken sempre em 1) e aos PIOs

B2 O Exemplo 2 no aplicativo

Faça

Num computador do laboratório, de ~/minha-conv:

```
python3 Python/morphe_app.py
```

Em *Placas do laboratório*, escolha a **sua** placa. No menu, **Convolução do aluno (conv_aluno × conv1d)** → **Carregar o Exemplo 2 do artigo** → **Comparar na FPGA**.

Confira

- o terceiro gráfico mostra $y = \{1, 4, 8, 8, 3, -2, -1\}$ com n de -1 a 5 : exatamente a Equação (17) e a Figura 1 do artigo. A janela sabe que h começa em $n = -1$ e põe a origem de y em $0 + (-1) = -1$;
- as hastes (seu módulo) e os $\times$ (o conv1d) coincidem, e o gráfico de baixo, a diferença, é zero;
- a barra de baixo diz *iguais, bit a bit*.

B3 Comparar com o conv1d em sinais de verdade

Faça

Gere sinais maiores no painel de cada lado e compare de novo. Sugestões:

$x[n]$	$h[n]$	o que mostra
senóide, 1024 amostras	degrau, 32 amostras	soma móvel de 32 amostras: y suaviza x
impulso	qualquer	$y = h$: a resposta ao impulso (Seção 2 do artigo)
degrau, 8	degrau, 8	um triângulo de 15 amostras
exponencial, 1024	senóide, 1024	o caso maior: 2047 amostras, ≈ 1 milhão de produtos

Confira

Em todos os casos, *iguais, bit a bit*, e diferença zero. Se não forem, a barra diz em que n está a primeira diferença e quantas amostras cada módulo acerta contra a mesma conta feita no NumPy — o que separa um erro seu de um erro do conv1d.

Pense

O conv1d percorre sempre as 2047 linhas de uma convolução 1024×1024 , com zeros onde não há sinal; o seu só faz as $n_x + n_h - 1$ linhas pedidas, e em cada uma só as casas coloridas. Quantos produtos cada um faz para $n_x = 1024$ e $n_h = 32$? E quantos ciclos o seu leva, a 4 ciclos por produto? A 50 MHz, quanto tempo?

B4 Devolver a placa à turma

Faça

Sempre, ao terminar. Na estação, de ~/minha-conv:

```
./morphe-up.sh --down
```

e depois, de /opt/morphe, com o mesmo CABO da A6:

```
cd /opt/morphe
```

```
./morphe-up.sh --cable 'CABO'
```

Erros comuns

Sintoma	Causa e solução
<i>IP upgrade recommended</i> , ou <i>Used ... 23.1</i>	abriu com outro Quartus; feche sem salvar e abra o 20.1
<i>can't find port</i> ao compilar	mudou o nome ou a largura de uma porta do <code>conv_aluno.v</code> ; volte à da A3
<i>Slack</i> negativo no <code>clock_50_1</code>	conta demais num ciclo só (multiplicar <i>e</i> somar no mesmo estado, por exemplo); separe em dois estados
o aplicativo diz <i>timeout</i>	o <code>done</code> não subiu em 5s: o módulo travou num estado. Simule (A5)
o aplicativo diz <i>opcode desconhecido</i>	o servidor da placa é o antigo: faltou o <code>--deploy</code> da A6, ou o app está ligado em outra placa
o aplicativo diz <i>bitstream ... nao e o deste servidor</i>	a placa está com outro <code>.sof</code> : grave o seu (A6)
<i>y</i> todo zero	o módulo não grava: <code>y_we</code> nunca sobe (o servidor zera <i>y</i> antes de cada pedido)
a placa some da rede	<code>devmem</code> antes de gravar o <code>.sof</code> , ou <code>hps_0.h</code> de outro <code>.sof</code>
nunca use System → Assign Base Addresses	muda o endereço de todos os componentes, e o servidor passa a escrever no lugar errado